

```
In [3]: # Import required modules
import matplotlib
import pandas as pd
import sqlite3
from openpyxl import load_workbook, Workbook
from openpyxl.styles import Font
from openpyxl.chart import LineChart, Reference, Series
import missingno as msno
from matplotlib import pyplot as plt
import seaborn as sns

In [9]: print('DATABASE CONNECTION ,LOAD DATA TO DATABASE FROM CSV FILE ,EXCEL CONNECTION ,FETCH DATA TO AND EXCEL ,DRAW LINECHART IN EXCEL')

Out[9]: DATABASE CONNECTION ,LOAD DATA TO DATABASE FROM CSV FILE ,EXCEL CONNECTION ,FETCH DATA TO AND EXCEL ,DRAW LINECHART IN EXCEL

In [5]: # Connecting to sqlite database
connection = sqlite3.connect('Netflix_Userbase.db')

# Creating a cursor object to execute SQL queries on a database table
cursor = connection.cursor()

# Drop the table if exist
drop_table='DROP TABLE IF EXISTS netflix_userbase_details;'
cursor.execute(drop_table)

# Table definition
create_table = '''
CREATE TABLE netflix_userbase_details
(
    User_ID            INTEGER      PRIMARY KEY NOT NULL
    ,Subscription_Type VARCHAR(255)
    ,Monthly_Revenue   INTEGER
    ,Join_Date         VARCHAR(255)
    ,Last_Payment_Date VARCHAR(255)
    ,Country           VARCHAR(255)
    ,Age               INTEGER
    ,Gender             VARCHAR(255)
    ,Device            VARCHAR(255)
    ,Plan_Duration     VARCHAR(255)
);
'''

# Creating the table into database
cursor.execute(create_table)

# Read data into panda series from the .csv file
insert_records=pd.read_csv('Netflix_Userbase.csv')

# SQL query to insert data into the table and replace object type column to int type
tempdict={}
templist=list(pd.unique(insert_records['Subscription Type']))
for i in range(len(templist)):
    tempdict[templist[i]]=i+1
insert_records['Subscription Type'].replace(tempdict,inplace=True)
insert_records.to_sql('netflix_userbase_details', connection, if_exists='replace', index=False)

# SQL query to retrieve all unique country data from table
country_list=[]
all_country = 'SELECT distinct Country FROM netflix_userbase_details limit 1'
cursor.execute(all_country)
for i in cursor.execute(all_country).fetchall():
    country_list.append(i)

#Create workbook for each unique country
for i in country_list:
    tempdata=connection.execute('SELECT * FROM netflix_userbase_details WHERE Country=?',(i)).fetchall()#User_ID,Country,Monthly_Revenue
    heading=connection.execute('SELECT * FROM netflix_userbase_details WHERE Country=?',(i)).description
    filename=str(list(i)[0])

    #Create new workbook
    wb = Workbook(filename+'.xlsx')
    wb.save(filename+'.xlsx')

    # Load present workbook,Worksheet object point to an existing worksheet & access cell data
    wb = load_workbook(filename =filename+'.xlsx',data_only=True )

    #If sheetname present remove
    if "Mydatasheet" in wb.sheetnames:
        wb.remove("Mydatasheet")
    ws_data = wb.active
    ws_data.title = "Mydatasheet"#"NoneType' object has no attribute 'append'

    #Heading & Font
    ws_data.append([heading[0][0],heading[1][0],heading[5][0],heading[2][0]])
    for col in ws_data.iter_cols(min_col=1,min_row=1,max_col=4, max_row=1):
        for c_cell in col:
            c_cell.font = Font(bold=True)

    #Write data to worksheet
    for element in tempdata:
        ws_data.append([element[0],element[1],element[5],element[2]])

    #Add new column
    new_col=ws_data.cell(row=1, column=5)#5th col
    new_col.value = 'New Monthly Revenue'
    new_col.font = Font(bold=True)
    for row_no in range(2,len(tempdata)+1+1):#len+head(1)+1
        ws_data.cell(row=row_no, column=5).value = ws_data.cell(row=row_no, column=4).value*ws_data.cell(row=row_no, column=2).value*1.2

    #If sheetname present remove
    if "MychartSheet" in wb.sheetnames:
        wb.remove('MychartSheet')
    ws_chart = wb.create_sheet('MychartSheet')

    #Create chart
    Lc = LineChart()
    Lc.title = 'Monthly Revenue Changes Line Chart For '+filename
    Lc.y_axis.title = 'Price'
    Lc.x_axis.title = 'Product Distribution'

    #Add data to chart
    data = Reference(ws_data, min_col=4, min_row=1, max_col=5, max_row=len(tempdata))
    Lc.add_data(data, titles_from_data=True)

    l1 = Lc.series[0]
    l1.marker.symbol = 'triangle'
    l1.marker.graphicalProperties.solidFill = '84CE66'
    l1.marker.graphicalProperties.line.solidFill = '84CE66'

    l2 = Lc.series[1]
    l2.graphicalProperties.line.dashStyle = 'dot'
    l2.marker.graphicalProperties.solidFill = 'D4CD3B'

    ws_chart.add_chart(Lc, 'A3')

    #Save workbook and close
    wb.save(filename+'.xlsx')
    wb.close()

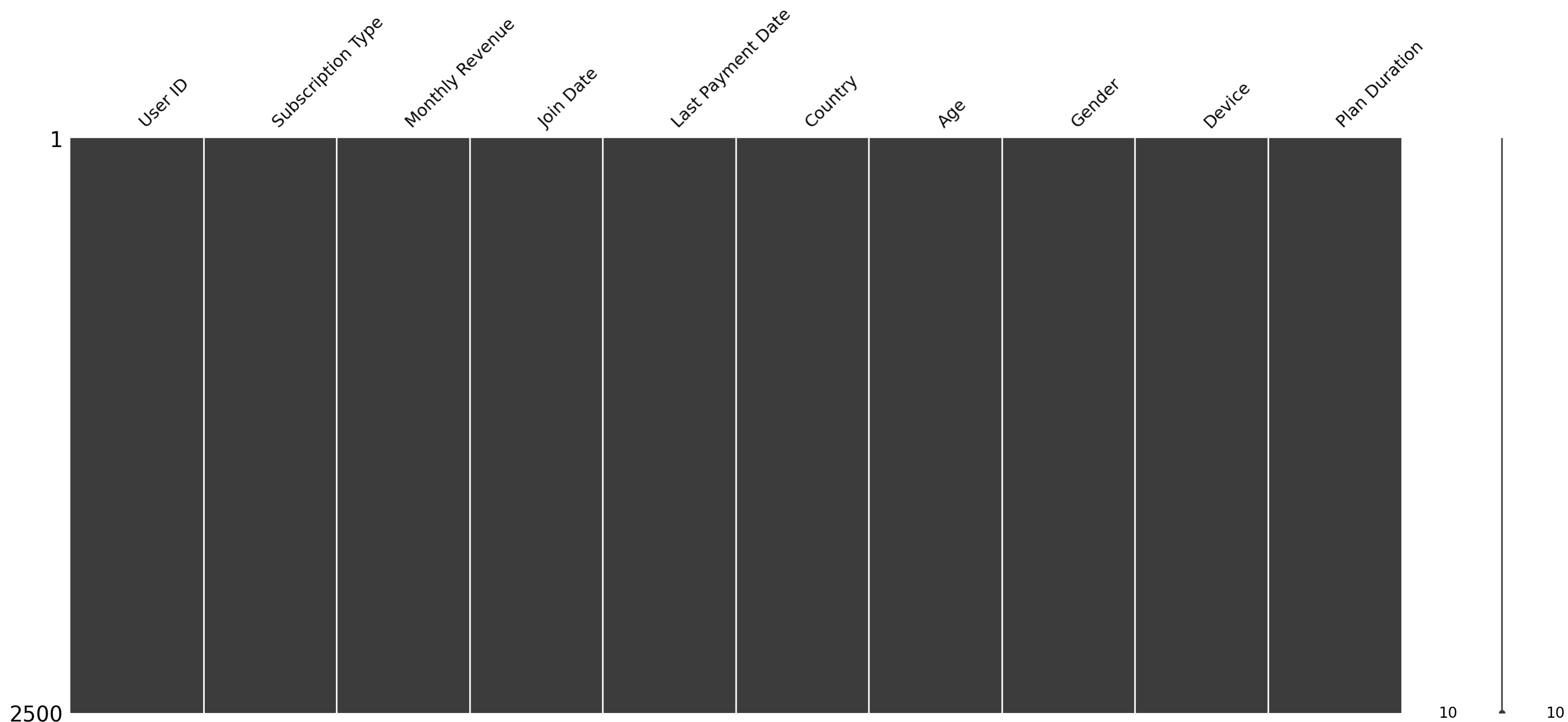
# Closing the database connection
connection.close()

In [10]: print('DATA VISUALIZATION')

Out[10]: DATA VISUALIZATION

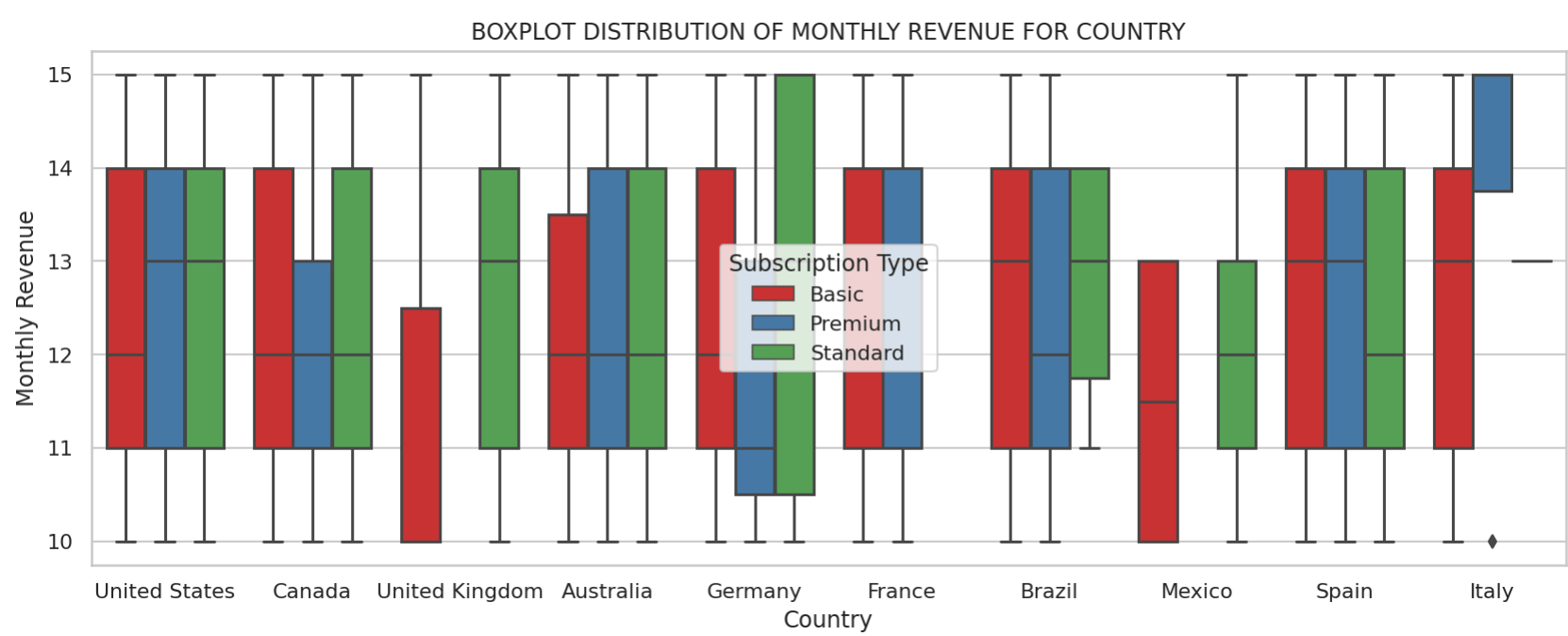
In [4]: #Load raw data in panda dataframe from visualization
data_records = pd.read_csv('Netflix_Userbase.csv')
#Check if dataframe in clean
msno.matrix(data_records)

Out[4]: <AxesSubplot: >
```

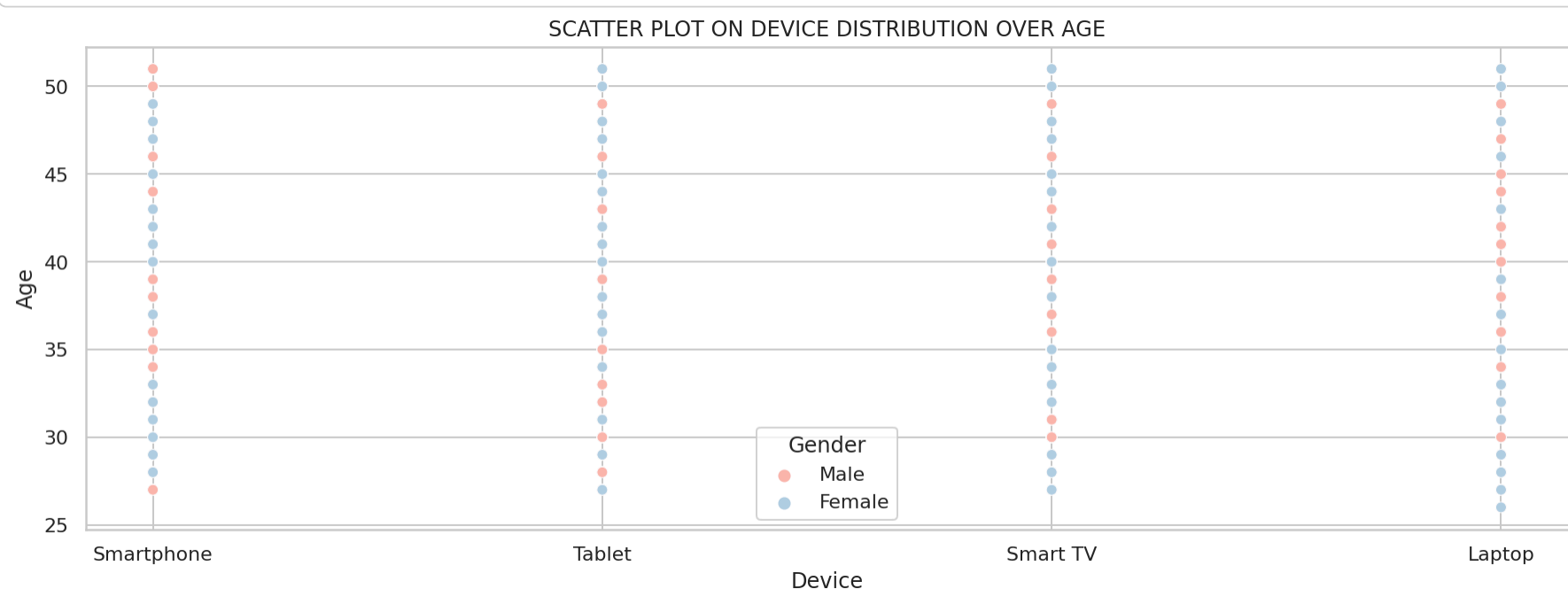


```
In [88]: # Boxplot distribution of 'Monthly Revenue' for each 'Country' based on 'Subscription Type'
fig, ax = plt.subplots(figsize=(14, 5))
ax.set_title('Boxplot distribution of Monthly Revenue for country'.upper())
sns.boxplot(data=data_records, x='Country', y='Monthly Revenue', hue='Subscription Type',ax=ax,palette="Set1")

Out[88]: <AxesSubplot: title='center': 'BOXPLOT DISTRIBUTION OF MONTHLY REVENUE FOR COUNTRY', xlabel='Country', ylabel='Monthly Revenue'>
```

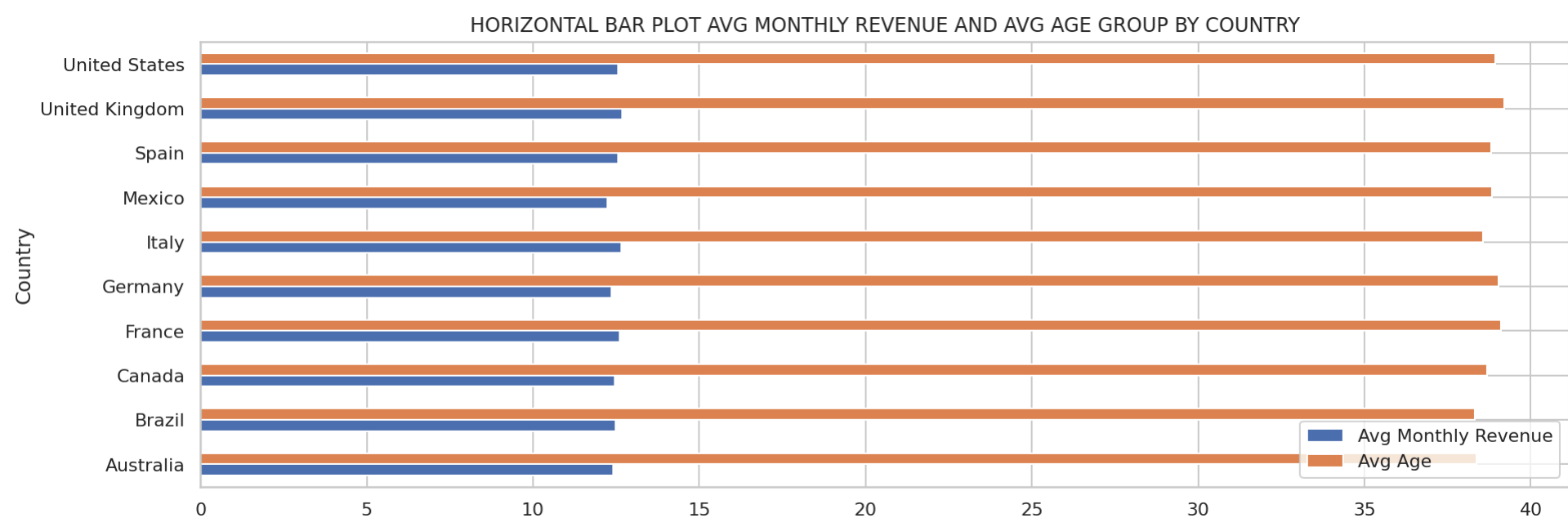


```
In [90]: # Scatter plot on 'Device' distribution over 'Age' partition by 'Gender'
fig, ax = plt.subplots(figsize=(15, 5))
ax.set_title('scatter plot on device distribution over age'.upper())
sns.scatterplot(data=data_records, x='Device', y='Age', hue='Gender',ax=ax,palette="Pastell1")
plt.show()
```



```
In [129]: # Horizontal bar plot 'Avg Monthly Revenue' and 'Avg Age' group by 'Country'
df = pd.DataFrame({'Avg Monthly Revenue': data_records.groupby(['Country']).describe()[['Monthly Revenue']]['mean'],
                  'Avg Age':data_records.groupby(['Country']).describe()[['Age']]['mean']},
                  index=data_records.groupby(['Country']).describe().index)
df.plot.barh(figsize=(15, 5),title='horizontal bar plot avg monthly revenue and avg age group by country'.upper())

Out[129]: <AxesSubplot: title='center': 'HORIZONTAL BAR PLOT AVG MONTHLY REVENUE AND AVG AGE GROUP BY COUNTRY', ylabel='Country'>
```



```
In [18]: #Heatmap of correlation on Dataframe numeric attribute
sns.heatmap(data_records[['Monthly Revenue','Age']].corr(), annot=True,cmap='PiYG')
```

Out[18]: <AxesSubplot: >

